Secure Code Debugging and Review Assistant

A Retrieval-Augmented Static-Analysis Tool for First-Pass Python Code Review

Field	Details
Institute / University	PCCOE (confirm official institute name)
Programme / Batch	Applied AI/ML Capstone — TechPulse FY-26
Student Name	Janvee Ghadge
PRN	123B1D060
Case Study ID	Case Study 4 (CS4)
Project Title	Secure Code Debugging and Review Assistant

Abstract

Manual code review is slow, and static-analysis tools, while good at flagging suspicious patterns, often leave developers without the context needed to understand and fix a finding. This project presents the Secure Code Debugging and Review Assistant, a Streamlit application that combines Python static analysis (Bandit and Ruff) with retrieval of secure-coding guidance from a local ChromaDB knowledge base, and exports the result as a downloadable Markdown report.

Guideline passages are embedded with sentence-transformers/all-MiniLM-L6-v2 and retrieved using finding-aware queries built from analyzer rule IDs, issues and code context. An optional OpenAI model can generate explanations; it was not configured in the evaluated run, so the system used a fallback summary. On a six-case manually recorded evaluation set, all cases were marked Pass. This is not a measured detection accuracy: precision, recall and latency were not calculated, and two of the cases share the same input. The tool is advisory and does not replace manual review, testing or a professional security audit.

Keywords: *secure code review, static analysis, Bandit, Ruff, retrieval-augmented generation, ChromaDB, Sentence Transformers, Streamlit*

Table of Contents

1. Introduction 4
   1.1 Background and Motivation 4
   1.2 Problem Statement 4
   1.3 Intended Users 4
   1.4 Objectives 4
   1.5 Scope and Non-Goals 4
2. Knowledge Base and Data 4
   2.1 Source and Format 4
   2.2 Preprocessing 4
   2.3 Chunking Strategy 4
3. System Design 5
   3.1 Architecture Overview 5
   3.2 Component Choices 5
4. Implementation 6
   4.1 Module Breakdown 6
   4.2 Review Workflow 6
   4.3 Explanation Stage 6
   4.4 Configuration and Privacy 6
5. Evaluation 7
   5.1 Methodology 7
   5.2 Test Cases and Results 7
   5.3 Metrics 7
   5.4 Reference: Bandit Rule IDs Observed 7
6. Results and Discussion 8

7. Responsible AI Measures 8
8. Innovation Highlights 8
9. Screenshots, Logs and Evidence ..... **Error! Bookmark not defined.**
10. Limitations, Challenges and Future Work 9
10.1 Challenges Encountered 9
10.2 Limitations 9
10.3 Future Work 9
11. Conclusion 9
12. AI-Tool Usage Declaration 9
13. Student Declaration 10
References and Tools ..... **Error! Bookmark not defined.**

1. Introduction

1.1 Background and Motivation

Manual code review can be time-consuming, and developers may overlook security weaknesses or code-quality issues. Static-analysis tools identify suspicious patterns but do not always provide enough contextual guidance for a developer to understand and remediate a finding.

1.2 Problem Statement

To design and prototype an accessible first-pass review tool that detects common security and quality issues in Python source code and pairs each finding with relevant secure-coding guidance and a reusable report.

1.3 Intended Users

Students, Python developers, reviewers and educators who need a first-pass review tool. The application is advisory and does not replace manual review, testing or a professional security audit.

1.4 Objectives

- Detect security and code-quality findings using Bandit, Ruff and the Python parser.
- Retrieve relevant secure-coding passages from a local, section-aware knowledge base.
- Present findings with severity, line number, rule ID, evidence and recommendation.
- Export a repeatable Markdown report that states its limitations explicitly.

1.5 Scope and Non-Goals

The review workflow is a Retrieval-Augmented Generation-style pipeline, not a multi-agent system. No autonomous agent orchestration or automatic code patching is claimed. Only Python source is supported.

2. Knowledge Base and Data

2.1 Source and Format

Item	Description
Source	Local Markdown/TXT files under data/guidelines/. Current demo source: coding_guidelines_demo.md.
Size and format	Small, locally maintained secure-coding guideline document in Markdown; indexed into 11 chunks in the observed run.

2.2 Preprocessing

- Text is read as UTF-8, with replacement for decoding errors.
- Markdown content is split into sections.
- Each section is normalised to whitespace-separated text before chunking.

2.3 Chunking Strategy

Section-aware chunking is used, with up to 180 words per chunk and no cross-section overlap in the updated implementation. Chunk metadata stores the source path and chunk number, which

allows the report to cite the origin of each retrieved passage. Headings such as Secure Subprocess Execution and Secrets and Credential Management stay attached to their passages.

3. System Design

3.1 Architecture Overview

The system is a linear five-stage pipeline: input, static analysis, guideline retrieval, explanation and report generation (Figure 1).

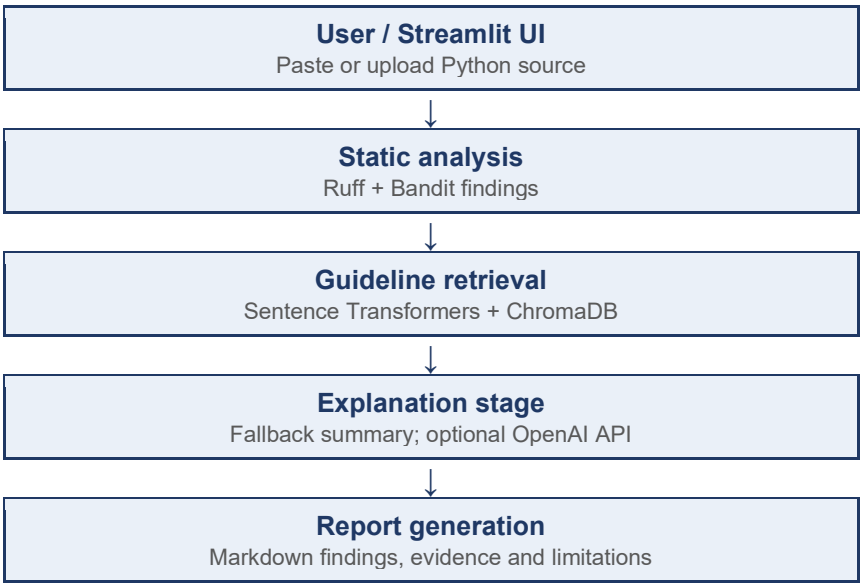


Figure 1. High-level architecture of the Secure Code Review Assistant.

3.2 Component Choices

Table 1. Technology stack

Component	Choice (name, version, hosting)
Pre-trained model	No general-purpose model is required for static analysis. An optional OpenAI model is set via OPENAI_MODEL; the default in code is gpt-4.1-mini (not tested in the current run).
Embedding model	sentence-transformers/all-MiniLM-L6-v2 via the Sentence Transformers embedding function.
Vector store	ChromaDB PersistentClient; local directory chroma_db; collection coding_guidelines.
RAG pipeline	Finding-aware search queries retrieve relevant local guideline passages. Not a multi-agent system.
Service layer and UI	Python application with Streamlit UI (app.py); report generation in report_generator.py.
Tools / deterministic rules	Bandit and Ruff for static analysis; deterministic guideline indexing and retrieval; Markdown report export.

4. Implementation

4.1 Module Breakdown

Table 2. Source modules and responsibilities

Module	Responsibility
app.py	Application entry point. Accepts Python source, runs analysis, retrieves guidelines, displays results and exposes report download.
review_engine.py	Invokes Ruff and Bandit when available and collects findings with tool name, rule ID, severity, line number, evidence and recommendation. Syntax errors are surfaced by the parser path.
rag_pipeline.py	Reads local .md/.txt guideline files, splits them into section-aware chunks, embeds and stores them in ChromaDB, and queries the collection. The index must be refreshed after guideline content or chunking logic changes.
report_generator.py	Produces the Markdown report with findings, evidence, guideline passages, fallback notice and limitations.

4.2 Review Workflow

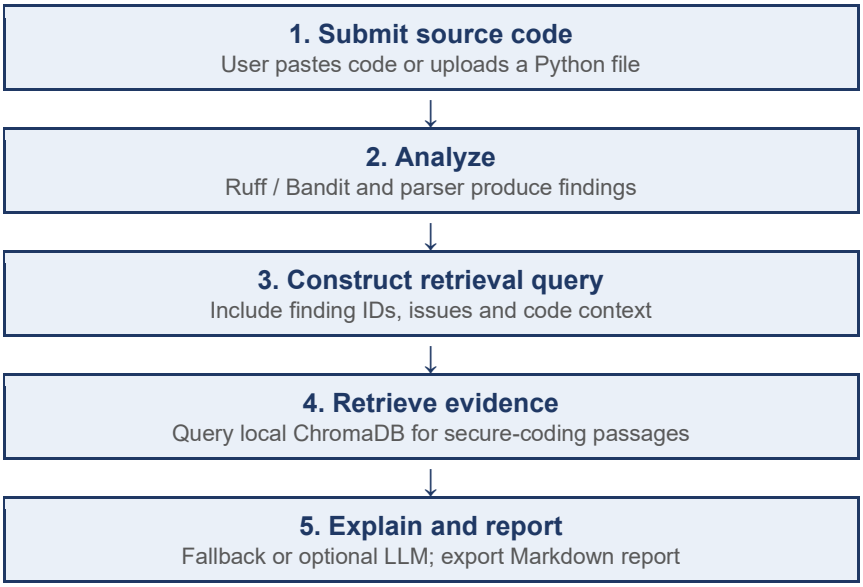


Figure 2. Workflow followed for each code review.

4.3 Explanation Stage

When OPENAI_API_KEY is absent, the system returns a fallback summary and explicitly notes that the LLM is not configured. When it is configured, the code can call the OpenAI Responses API to generate the explanation.

4.4 Configuration and Privacy

The optional API key and model are loaded from a local .env file. The .env file, real credentials and proprietary source code must not be committed. The application was tested locally using Python 3.11 and Streamlit.

5. Evaluation

5.1 Methodology

The evaluation set contains six manually recorded scenarios. Results reflect the observed local runs and the supplied evaluation CSV (evaluation/test_cases.csv). TC01 and TC02 reuse the same vulnerable sample, so they are not independent test inputs.

5.2 Test Cases and Results

Table 3. Test-case outcomes

ID	Scenario	Expected behaviour	Result
TC01	Hardcoded demo credential and risky subprocess call	Detect B105, B404 and B602 in sample_vulnerable.py	Pass — 3 findings recorded
TC02	Subprocess shell usage	Detect Bandit B602 for shell=True	Pass — B602 reported; overlaps TC01
TC03	Clean calculate_total function	No findings for the simple validated function	Pass — 0 findings
TC04	Invalid Python syntax	Report missing colon / syntax error at line 1	Pass — 1 syntax finding
TC05	Safer subprocess with shell=False	B602 absent; other analyzer warnings may remain	Pass — B404, B607 and B603 reported
TC06	Allowlisted user role	No static-analysis findings for allowlisted role sample	Pass — 0 findings

5.3 Metrics

Table 4. Metrics and observations

Metric	Score / observation
Test-case pass rate	6/6 recorded cases marked Pass. This is not a measured detection accuracy.
Retrieval relevance	For the vulnerable sample, retrieved evidence included credential guidance (0.67), subprocess guidance (0.67) and cryptography guidance (0.55). The third passage was less directly relevant.
Latency	Not formally measured in the current evaluation.
Precision / recall	Not calculated; a larger labelled dataset and ground-truth labels are required.

5.4 Reference: Bandit Rule IDs Observed

Table 5. Bandit rules appearing in the evaluation

Rule ID	Meaning (Bandit)	Seen in
B105	Hardcoded password string	TC01
B404	Import of the subprocess module	TC01, TC05
B602	subprocess call with shell=True	TC01, TC02

Rule ID	Meaning (Bandit)	Seen in
B603	subprocess call without shell (review of untrusted input)	TC05
B607	Starting a process with a partial executable path	TC05

In the vulnerable-code run, Bandit reported B404 (Low), B105 (Low) and B602 (High).

6. Results and Discussion

Bandit identified a hardcoded demo credential and the use of shell=True, while a simple clean function and an allowlisted-role sample returned no findings. The syntax-error case was surfaced through the parser path. In the safer-subprocess case, B602 was correctly absent while B404, B607 and B603 remained, consistent with the expectation that other analyzer warnings may persist.

Section-aware chunking and finding-aware retrieval improved the relevance of evidence for the vulnerable sample: credential and subprocess passages scored 0.67 each, while the cryptography passage (0.55) was less directly relevant, indicating room to improve retrieval ranking. These observations come from a small set of manually recorded cases and should not be generalised.

7. Responsible AI Measures

Table 6. Responsible AI measures adopted

Area	Measure adopted
Grounding and citations	Reports include source file and chunk identifiers for retrieved guideline passages. Retrieval evidence is advisory and should be checked against the actual code.
Privacy	Guideline storage and retrieval are local. The optional OpenAI API may transmit submitted code to an external provider; confidential or proprietary code must not be submitted without authorization.
Security	API keys are kept in .env and excluded from version control; any credential that may have been exposed should be rotated. Analyzer output is not proof of exploitability.
Human oversight	Developers must inspect the evidence, validate fixes and run tests. AI-generated content is not claimed to be tested unless test evidence exists.

8. Innovation Highlights

- **Finding-aware retrieval queries:** The retrieval query can include analyzer rule IDs and recommendations, rather than relying only on raw source code.
- **Section-aware guideline chunks:** Headings are retained with their passages to improve retrieval relevance.
- **Evidence-oriented reports:** Findings are exported with severity, line number, rule ID, code evidence, guideline passages and explicit limitations.

These are implementation features demonstrated in the current prototype; no autonomous agent orchestration or automatic code patching is claimed.

10. Limitations, Challenges and Future Work

10.1 Challenges Encountered

- Configuring a compatible Python environment.
- Maintaining the local ChromaDB index after changing the chunking logic.
- Tuning retrieval so that guidance matches the detected issue.

10.2 Limitations

- The optional LLM was unconfigured in the evaluated run.
- Retrieval metrics and latency have not been formally measured.
- The six-case evaluation set is small, and TC01 and TC02 share an input.
- The knowledge base is a single small guideline document (11 chunks).

10.3 Future Work

- Configure and evaluate the LLM explanation stage.
- Expand labelled test data and calculate precision and recall.
- Improve retrieval ranking.
- Add automated regression tests.

11. Conclusion

The prototype demonstrates a practical combination of deterministic static analysis and retrieval of secure-coding guidance. Testing showed that Bandit identified a hardcoded demo credential and the use of shell=True, while a simple clean function returned no findings. Section-aware chunking and finding-aware retrieval improved the relevance of evidence for the vulnerable sample, and the downloadable Markdown report provides a repeatable artifact for review and evaluation. Further evaluation on a larger labelled dataset is needed before drawing conclusions about detection quality.

12. AI-Tool Usage Declaration

Table 7. AI tools used

AI tool / model	Purpose	Artifact affected	Verified by student (Y/N)
ChatGPT / OpenAI assistant	Drafting, debugging guidance, report wording and retrieval troubleshooting	Code changes and this technical report; student must review and verify	_____
Sentence Transformers: all-MiniLM-L6-v2	Embedding guideline chunks and retrieval queries	Local RAG retrieval pipeline	Y (local retrieval tested)
OpenAI API model (optional)	LLM-generated review explanation when configured	Optional explanation function	N — not configured in observed test

References and Tools

- Bandit — Python security linter
- Ruff — Python linter
- ChromaDB — open-source embedding database
- Sentence Transformers — all-MiniLM-L6-v2 embedding model
- Streamlit — Python web application framework
- OpenAI API — Responses API (optional component)

13. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation. I will review and correct all generated content, verify technical claims against my code and test evidence, and complete the AI-tool usage declaration accurately before submission.

Name: Janvee Ghadge PRN: 123B1D060

Signature: Date: